Evidencia de aprendizaje 2: Creación de repositorio

Procesamiento funcional de datos para el sistema de gestión de transporte urbano TecnoMovil Data

Jorge Daniel Londoño Duque

Ingeniería de Software y Datos

Facultad de Ingeniería y Ciencias Agropecuarias

Universidad Digital de Antioquia

PREICA2602B020047: Programación Orientada a Objetos II

Docente: Leonardo Montes

Medellín, 1 de noviembre de 2026

Introducción

La Alcaldía de TecnoValle moviliza más de 450.000 pasajeros al día y recibe entre 6 y 12 millones de registros de validadores, sensores y aplicaciones. Su plataforma actual, imperativa y con estructuras mutables compartidas, tarda hasta dos horas en procesar un día y presenta errores de conteo por condiciones de carrera. Los ingenieros del proyecto TecnoMovil Data señalaron como módulo más crítico el de procesamiento y agregación de datos diarios.

Este trabajo rediseña ese módulo en Java, sin frameworks externos, con el paradigma funcional. Los errores que describe el caso nacen de que varias partes del programa modifican los mismos datos; si ninguna función modifica nada, esa clase de error deja de ser posible y el procesamiento se puede repartir entre varios hilos sin sincronización adicional. Como afirma Bloch (2018), las funciones usadas en un flujo deben estar libres de efectos secundarios para que este paradigma aporte su beneficio.

El documento presenta los objetivos, el diseño de la solución, la explicación de cómo se aplican las funciones puras, la inmutabilidad, las funciones de orden superior, las lambdas y los Streams, las evidencias de ejecución, las conclusiones y el enlace al repositorio. El código fuente completo está en el Anexo A.

Objetivos

Objetivo general

Diseñar e implementar en Java un módulo funcional que procese y agregue los datos diarios del sistema de transporte de TecnoValle mediante Streams, lambdas, funciones puras y colecciones inmutables, y alojarlo en un repositorio de GitHub.

Objetivos específicos

- Aplicar los conceptos de la programación funcional en un escenario realista: las seis tareas de análisis que plantea el caso de estudio.
- Manejar grandes volúmenes de datos de manera declarativa, con pruebas de al menos 6 millones de registros simulados.
- Experimentar con funciones puras, lambdas y operaciones de orden superior, y comprobar que el mismo código da resultados idénticos con `stream()` y `parallelStream()`.
- Crear un repositorio en GitHub y alojar la solución en su rama principal.

Desarrollo

Análisis de los requerimientos

Cada registro contiene `idUsuario`, `ruta`, `estacion`, `accion` y `timestamp`. La Tabla 1 relaciona cada tarea del caso con el método que la resuelve y las operaciones funcionales que emplea.

Tabla 1

Tareas del caso de estudio y su implementación

Tarea	Método de AnalisisTransporte	Operaciones principales
a) Afluencia por estación	afluenciaPorEstacion, usuariosUnicosPorEstacion	filter, groupingBy, counting, mapping, toSet
b) Horas pico	flujoPorHora, horasPico	groupingBy, counting, sorted, limit
c) Rutas más utilizadas	rutasMasUtilizadas	filter, groupingBy, counting, sorted
d) Patrones de viaje	patronesDeViaje	groupingBy, mapping, toUnmodifiableList
e) Tiempo entre estaciones	tiemposPorTramo, minutosPromedioGeneral	groupingBy, flatMap, IntStream.range, summarizingLong, average
f) Sobrecarga en rutas	detectarSobrecarga	groupingBy anidado, max, función clasificadora

El enunciado deja algunos puntos abiertos. La Tabla 2 recoge cómo se interpretó cada uno y por qué.

Tabla 2

Decisiones de interpretación

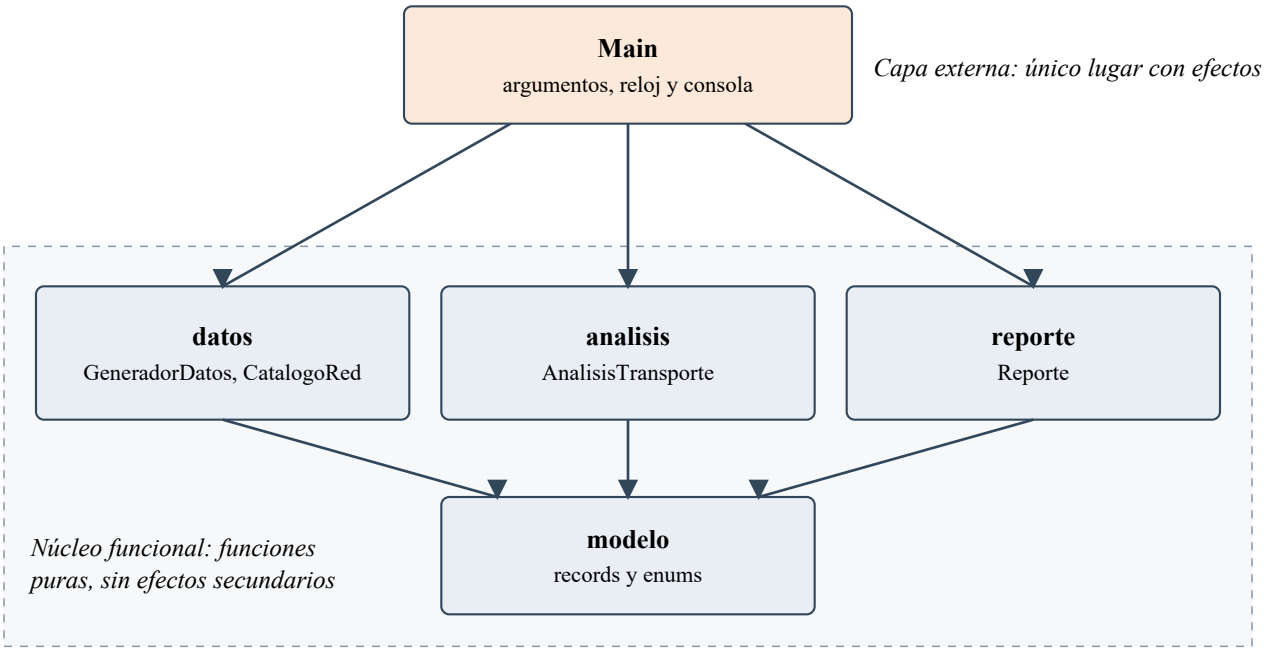
Punto del enunciado	Decisión	Justificación
«Cuántos usuarios ingresan» (a)	Se calculan los ingresos y también los usuarios distintos.	Un usuario que entra dos veces a la misma estación genera dos ingresos, pero es una sola persona; cada cifra sirve a una decisión diferente.
«Momentos de mayor flujo» (b)	El flujo cuenta entradas y salidas.	Ambas acciones representan personas pasando por las estaciones en esa hora.
«Volumen de uso» (c)	Se cuentan solo las entradas.	Cada viaje genera una entrada y una salida; contar ambas duplicaría cada uso.
«Estaciones consecutivas» (e)	Solo se mide una entrada seguida de una salida en otra estación.	De una salida a la siguiente entrada pueden pasar horas de trabajo o estudio; incluirlas falsearía el promedio.
«Umbral de ocupación (simulado)» (f)	Ocupación = pasajeros de la hora de mayor demanda / capacidad horaria simulada. Crítica por encima de 90 %.	Una ruta se satura en su peor hora; el promedio del día ocultaría esa saturación.

Diseño de la solución

Se separó un núcleo de funciones puras de una capa externa mínima. La clase `Main` es la única que lee argumentos, consulta el reloj e imprime en consola; todo lo demás recibe valores y devuelve valores. La Figura 1 muestra esta organización.

Figura 1

Arquitectura del módulo TecnoMovil Data



Nota. Las flechas indican dependencia entre paquetes. Elaboración propia.

Los datos provienen de `GeneradorDatos`, que simula usuarios con viajes de ida y regreso concentrados en la mañana y la tarde. Con 450.000 usuarios, la cifra diaria del caso, genera cerca de 1,8 millones de registros; con 1.500.000 usuarios supera los 6 millones. Cada viaje obtiene su propio generador pseudoaleatorio a partir de una semilla derivada, de modo que la misma semilla siempre produce los mismos datos, incluso en paralelo.

Aplicación del paradigma funcional

Funciones puras

Una función pura devuelve siempre el mismo resultado para los mismos parámetros y no altera nada fuera de ella. Las seis tareas se implementan así. Por ejemplo, la regla que decide si dos registros consecutivos forman un desplazamiento solo depende de sus dos parámetros:

```
private static boolean esDesplazamiento(RegistroTransporte actual, RegistroTransporte siguiente) {
    return actual.esEntrada()
            && !siguiente.esEntrada()
            && !actual.estacion().equals(siguiente.estacion());
}
```

Incluso los textos del reporte son funciones puras: `Reporte` devuelve cadenas y `Main` las imprime. La verificación llama dos veces a las tareas con la misma entrada y compara ambos resultados (Figura 6).

Inmutabilidad

La inmutabilidad se aplica en tres niveles. Los registros y resultados son `record`, cuyos campos son finales (Oracle, s.f.-e; Bloch, 2018). Ninguna operación modifica la lista de entrada: cuando se necesita un orden distinto, `sorted` produce una lista nueva. Y los resultados se entregan como colecciones no modificables, de modo que quien las recibe tampoco puede alterarlas:

```
return flujo(registros)
        .filter(filtro)
        .collect(Collectors.collectingAndThen(
                Collectors.groupingBy(clave, TreeMap::new, Collectors.counting()),
                Collections::unmodifiableMap));
```

Durante la agrupación, el colector usa un mapa intermedio interno, pero esa estructura no es accesible desde fuera ni la comparten varios hilos, pues cada uno acumula en su propio contenedor y luego se combinan (Oracle, s.f.-a). La verificación intenta modificar los resultados y confirma que Java lo rechaza con `UnsupportedOperationException`.

Funciones de orden superior

Son funciones que reciben o devuelven otras funciones. `contarPor` recibe el filtro y la clave de agrupación como parámetros, y por eso las tareas a, b y c la reutilizan en lugar de repetir el mismo código. `clasificador` hace lo contrario: recibe un umbral y devuelve la función que clasifica una ocupación:

```
static DoubleFunction<NivelOcupacion> clasificador(double umbral) {
    return ocupacion -> ocupacion > umbral ? NivelOcupacion.CRITICA : NivelOcupacion.NORMAL;
}
```

Del mismo modo, `detectarSobrecarga` recibe la capacidad como `ToIntFunction<String>`: no le importa de dónde sale ese dato, y la verificación puede pasarle una capacidad distinta a la de producción sin cambiar el método.

Lambdas y composición

Las lambdas expresan el comportamiento como un valor que se pasa a otra función (Oracle, s.f.-b). Se usan en forma de expresión, como `registro -> true`, y de referencia a método, como `RegistroTransporte::estacion`. También se componen funciones para formar reglas más grandes a partir de otras pequeñas, como este criterio de orden:

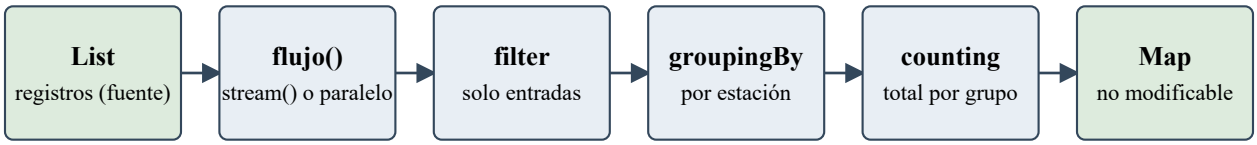
```
Comparator.comparingDouble(TiempoTramo::minutosPromedio).reversed()
        .thenComparing(TiempoTramo::origen)
        .thenComparing(TiempoTramo::destino)
```

Streams

Un Stream describe qué se quiere calcular y no cómo recorrer los datos: no hay ciclos, contadores ni variables acumuladoras (Urma et al., 2018). La Figura 2 muestra el pipeline de la tarea a.

Figura 2

Pipeline declarativo de la afluencia por estación



La lista original no cambia: cada paso produce un valor nuevo a partir del anterior.
Las lambdas solo leen el registro que reciben; por eso el flujo se puede dividir entre varios hilos.

Nota. Elaboración propia.

La tarea e es la más elaborada. `IntStream.range` recorre las posiciones de los registros de cada usuario para comparar cada uno con el siguiente, y `flatMap` une los desplazamientos de todos los usuarios en un solo flujo:

```
return IntStream.range(0, ordenados.size() - 1)
        .filter(i -> esDesplazamiento(ordenados.get(i), ordenados.get(i + 1)))
        .mapToObj(i -> crearDesplazamiento(ordenados.get(i), ordenados.get(i + 1)));
```

Paralelización

La documentación de Java exige que las operaciones de un flujo no interfieran con la fuente y, en general, no dependan de un estado que cambie durante la ejecución (Oracle, s.f.-c). Como el módulo cumple esas condiciones, cambiar de modo solo requiere elegir `AnalisisTransporte.enParalelo()` en lugar de `secuencial()`:

```
private <T> Stream<T> flujo(Collection<T> datos) {
    return paralelo ? datos.parallelStream() : datos.stream();
}
```

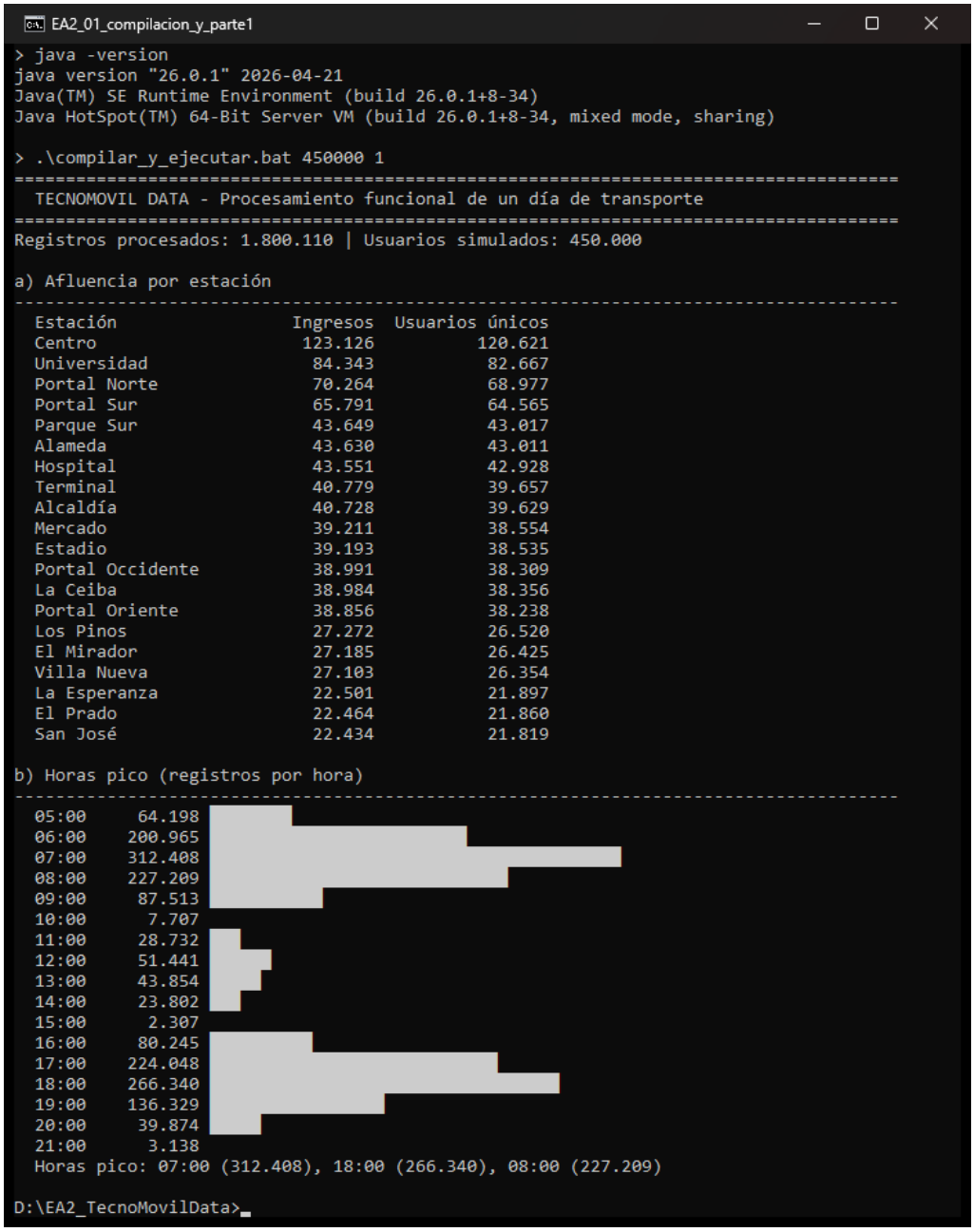
Los patrones de viaje conservan el orden de los registros también en paralelo, porque la lista de origen es ordenada y `groupingBy` no es un colector concurrente (Oracle, s.f.-a; Oracle, s.f.-d).

Evidencias de ejecución

El proyecto se compiló con el JDK 26.0.1 con la opción `--release 17`, por lo que también funciona desde Java 17. Las capturas son de ejecuciones reales en consola de Windows. La Figura 3 muestra las tareas a y b, y la Figura 4, las tareas c a f con 1.800.110 registros.

Figura 3

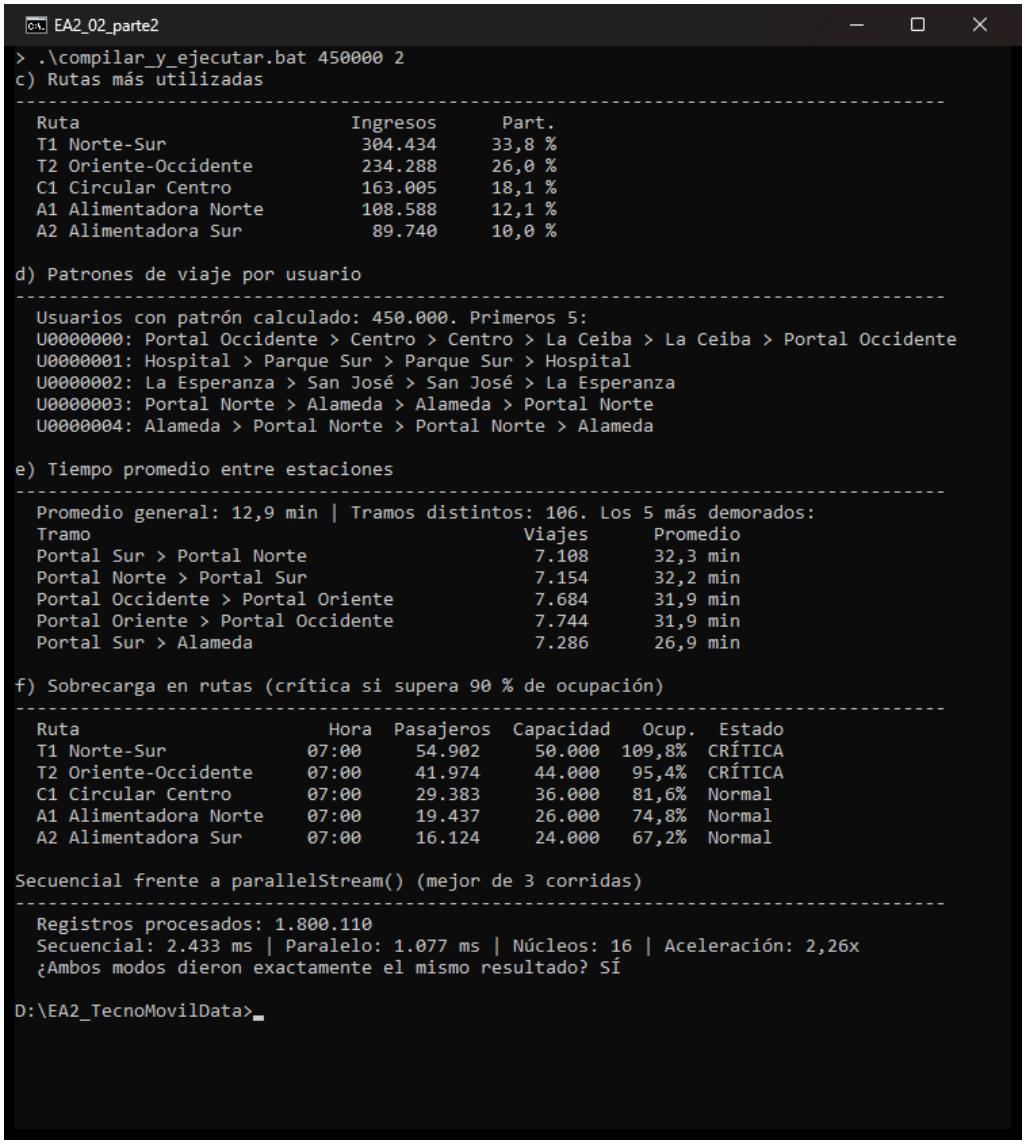
Compilación, ejecución y resultados de las tareas a y b



Nota. Captura de pantalla propia.

Figura 4

Resultados de las tareas c, d, e y f con 1,8 millones de registros



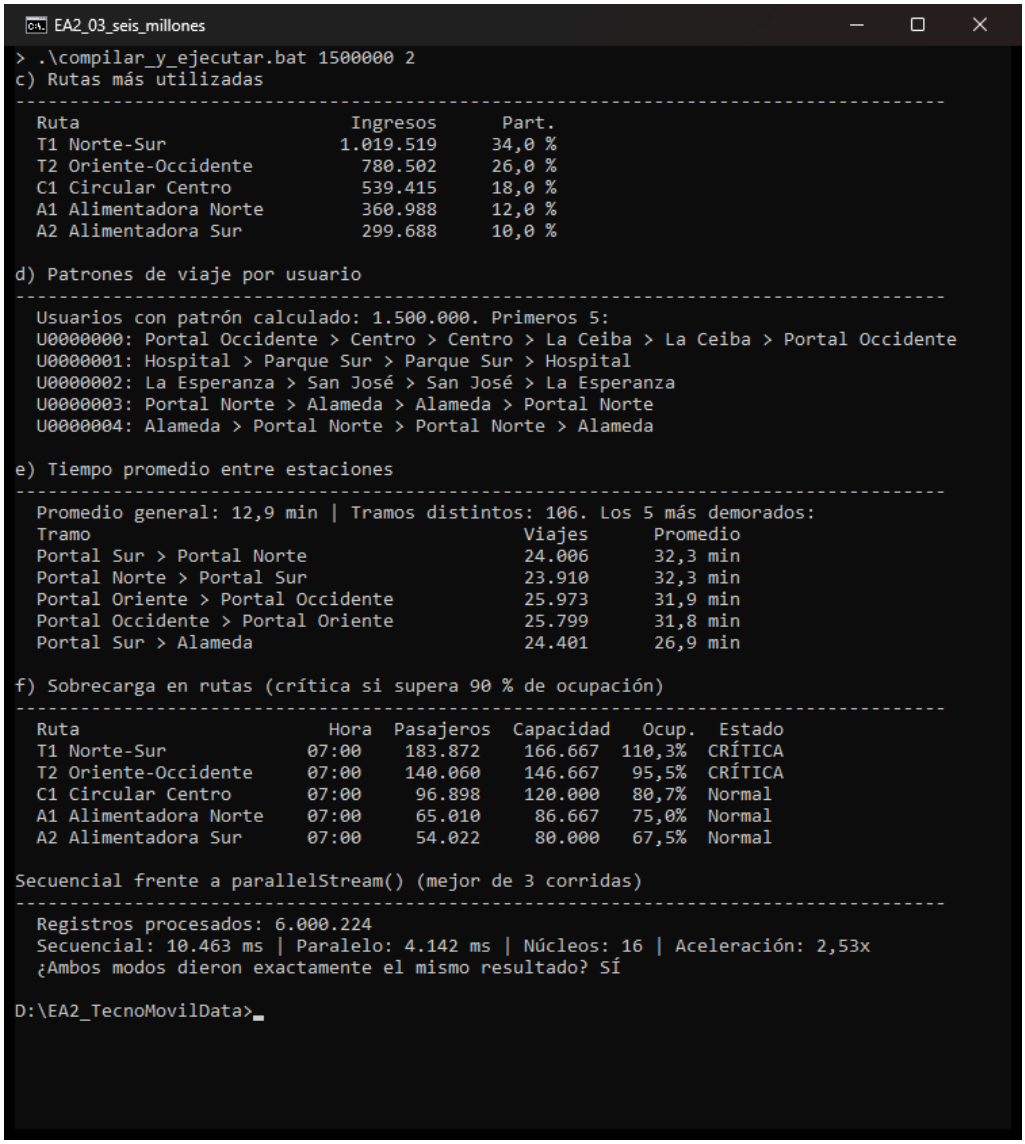
Nota. Captura de pantalla propia.

Los resultados son coherentes con los datos simulados: hay picos a las 7:00 y a las 18:00, y Centro lidera la afluencia porque tres rutas pasan por ella. Las rutas T1 Norte-Sur (109,8 %) y T2 Oriente-Occidente (95,4 %) quedan marcadas como críticas. Estas cifras demuestran que el módulo funciona, pero no describen una ciudad real, porque tanto la demanda como la capacidad son simuladas. La suma de los ingresos por ruta (900.055) es exactamente la mitad de los registros, como corresponde a un viaje con una entrada y una salida.

Para comprobar el manejo de grandes volúmenes se repitió la ejecución con 1.500.000 usuarios (Figura 5). La Tabla 3 compara los tiempos de las seis tareas en ambos modos.

Figura 5

Ejecución con más de 6 millones de registros



Nota. Captura de pantalla propia.

Tabla 3

Tiempo de las seis tareas: secuencial frente a paralelo

Registros	Secuencial (ms)	Paralelo (ms)	Aceleración	Resultados idénticos
1.800.110	2.433	1.077	2,26x	Sí
6.000.224	10.463	4.142	2,53x	Sí

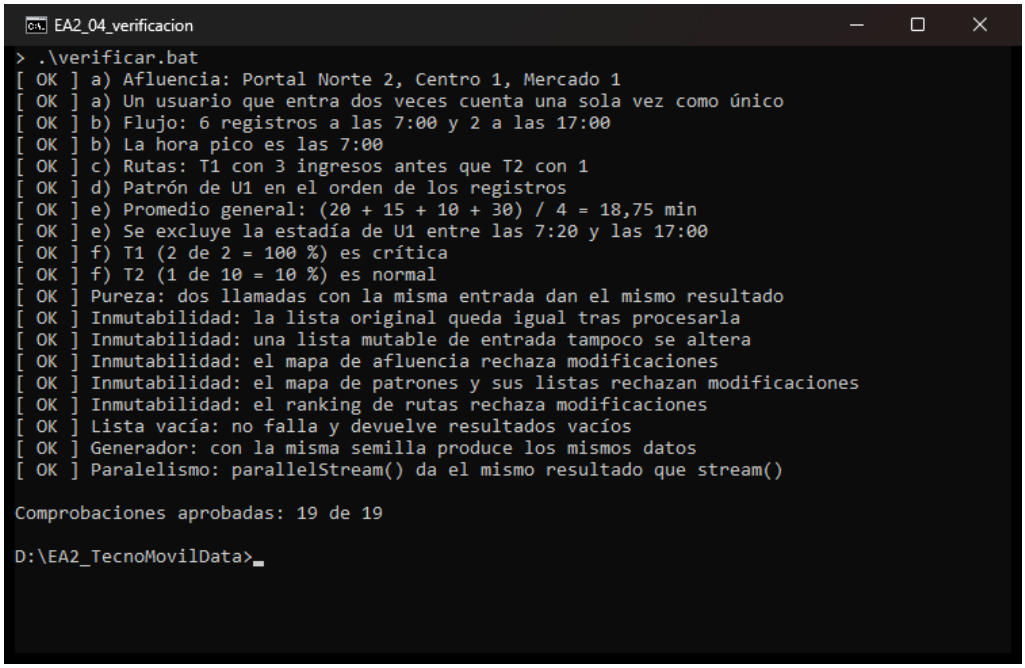
Nota. Mejor de tres corridas en un equipo con 16 núcleos lógicos. Los tiempos varían entre ejecuciones.

El paralelismo redujo el tiempo a menos de la mitad, aunque lejos de una aceleración de 16 veces. Esto es esperable: dividir los datos y combinar los mapas parciales tiene un costo, y el paralelismo no garantiza una mejora proporcional (Oracle, s.f.-d; Bloch, 2018). Lo más importante para el caso es que ambos modos dieron exactamente el mismo resultado, es decir, sin los errores de conteo por condiciones de carrera. Procesar 6 millones de registros en unos 4 segundos no se puede comparar directamente con las dos horas del sistema actual, ya que ese sistema también lee fuentes reales y corre en otro equipo; solo indica que el diseño tiene margen para ese volumen.

Por último, `VerificacionFunciones` compara los resultados con un conjunto pequeño calculado a mano y comprueba pureza, inmutabilidad, listas vacías, reproducibilidad y equivalencia entre modos, sin usar frameworks de pruebas. Las 19 comprobaciones se aprobaron (Figura 6).

Figura 6

Comprobaciones de correctitud, pureza e inmutabilidad

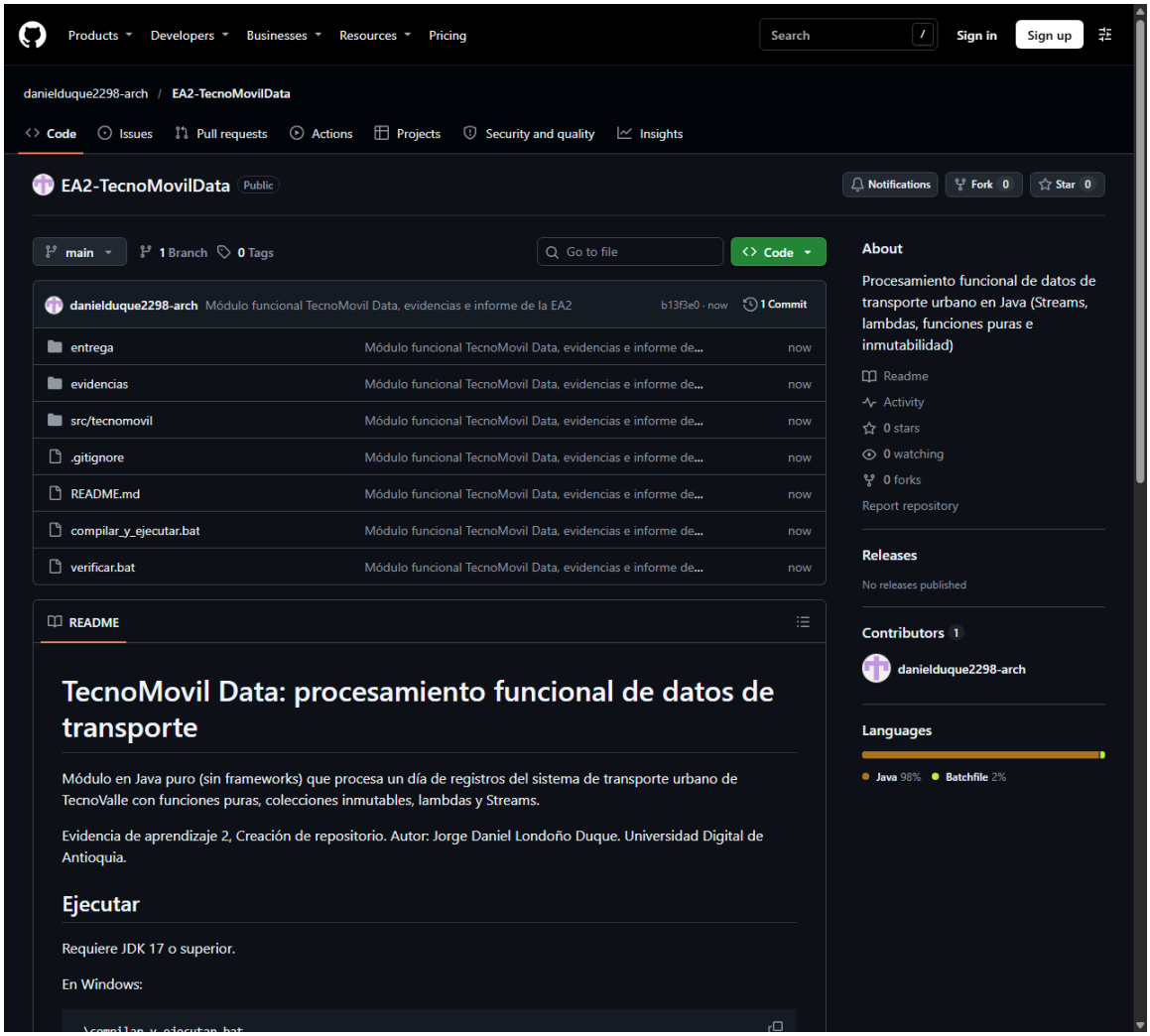


Nota. Captura de pantalla propia.

El repositorio se publicó en GitHub, con la solución en la rama principal (Figura 7).

Figura 7

Repositorio del proyecto en GitHub



Nota. Captura de pantalla propia.

Archivo ZIP del proyecto final

El archivo `TecnoMovilData_proyecto.zip` contiene el código fuente, los scripts de ejecución, las capturas y el README. Viene adjunto dentro de este PDF (panel de archivos adjuntos del lector) y también está en la carpeta `entrega` del repositorio.

Conclusiones

Las seis tareas del caso se resolvieron con funciones puras, lambdas, Streams y colecciones inmutables, y sus resultados coincidieron con los valores calculados a mano. Con ello se cumple el objetivo de aplicar el paradigma funcional en un escenario realista.

El enfoque declarativo manejó volúmenes grandes sin cambiar el código: el módulo procesó más de 6 millones de registros, el mínimo diario que plantea el caso, en cerca de 10 segundos en secuencial y 4 en paralelo.

La inmutabilidad ataca la causa de los errores de conteo del sistema actual. Al no existir estado compartido que modificar, `stream()` y `parallelStream()` dieron resultados idénticos. El paralelismo redujo el tiempo entre 2,3 y 2,5 veces, lo que muestra que es útil pero no proporcional al número de núcleos.

Las funciones de orden superior redujeron la duplicación: una misma función de conteo sirve para tres tareas al recibir su criterio como parámetro. Además, separar el núcleo puro de la consola facilitó verificar cada tarea por separado.

La principal limitación es que los datos y las capacidades son simulados, por lo que las rutas críticas y las horas pico ilustran el funcionamiento del módulo y no la situación real de una ciudad. Antes de integrarlo a TecnoMovil Data haría falta conectarlo a las fuentes reales y repetir las mediciones.

La solución quedó alojada en la rama principal de un repositorio de GitHub, junto con este informe y el archivo ZIP.

Enlaces del repositorio

Tabla 4

Repositorio por integrante

Integrante	Repositorio en GitHub
Jorge Daniel Londoño Duque	https://github.com/danielduque2298-arch/EA2-TecnoMovilData

Referencias

Bloch, J. (2018). *Effective Java* (3.ª ed.). Addison-Wesley.

Oracle. (s.f.-a). *Collectors (Java SE 17 & JDK 17)*. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/stream/Collectors.html

Oracle. (s.f.-b). *Lambda expressions*. The Java Tutorials. https://docs.oracle.com/javase/tutorial/java/javaOO/lambdaexpressions.html

Oracle. (s.f.-c). *Package java.util.stream (Java SE 17 & JDK 17)*. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/stream/package-summary.html

Oracle. (s.f.-d). *Parallelism*. The Java Tutorials. https://docs.oracle.com/javase/tutorial/collections/streams/parallelism.html

Oracle. (s.f.-e). *Record classes*. Java Language Updates. https://docs.oracle.com/en/java/javase/17/language/records.html

Urma, R.-G., Fusco, M., y Mycroft, A. (2018). *Modern Java in action: Lambdas, streams, functional and reactive programming*. Manning.

Anexo A

Código fuente completo

src/tecnomovil/modelo/Accion.java

```
package tecnomovil.modelo;

/** Tipo de validación que genera un registro. */
public enum Accion {
    /** El usuario ingresa a la estación. */
    ENTRADA,
    /** El usuario valida para salir o para cambiar de ruta. */
    SALIDA
}
```

src/tecnomovil/modelo/RegistroTransporte.java

```
package tecnomovil.modelo;

import java.time.LocalDateTime;
import java.util.Objects;

/**
 * Registro inmutable de una validación del sistema de transporte (requerimiento 3.1).
 * Al ser un record, todos sus campos son finales y no existen métodos que los modifiquen.
 */
public record RegistroTransporte(
        String idUsuario,
        String ruta,
        String estacion,
        Accion accion,
        LocalDateTime timestamp) {

    public RegistroTransporte {
        Objects.requireNonNull(idUsuario, "idUsuario");
        Objects.requireNonNull(ruta, "ruta");
        Objects.requireNonNull(estacion, "estacion");
        Objects.requireNonNull(accion, "accion");
        Objects.requireNonNull(timestamp, "timestamp");
    }

    public boolean esEntrada() {
        return accion == Accion.ENTRADA;
    }

    public int hora() {
        return timestamp.getHour();
    }
}
```

src/tecnomovil/modelo/Conteo.java

```
package tecnomovil.modelo;

/** Par inmutable clave-total usado en los rankings (horas pico y rutas más utilizadas). */
public record Conteo<K>(K clave, long total) {
}
```

src/tecnomovil/modelo/TiempoTramo.java

```
package tecnomovil.modelo;

/** Tiempo promedio que toman los usuarios en moverse de una estación a otra. */
public record TiempoTramo(String origen, String destino, long viajes, double minutosPromedio) {
}
```

src/tecnomovil/modelo/NivelOcupacion.java

```
package tecnomovil.modelo;

/** Clasificación de una ruta según su ocupación en la hora de mayor demanda. */
public enum NivelOcupacion {
    NORMAL,
    CRITICA
}
```

src/tecnomovil/modelo/EstadoRuta.java

```
package tecnomovil.modelo;

/**
 * Resultado de evaluar la ocupación de una ruta.
 * La ocupación es la razón entre los pasajeros de la hora de mayor demanda y la capacidad horaria.
 */
public record EstadoRuta(
        String ruta,
        int horaMaxima,
        long pasajerosHoraMaxima,
        int capacidadHoraria,
        double ocupacion,
        NivelOcupacion nivel) {
}
```

src/tecnomovil/modelo/ResumenDiario.java

```
package tecnomovil.modelo;

import java.util.List;
import java.util.Map;

/** Agrupa, de forma inmutable, las respuestas a las seis tareas del caso de estudio. */
public record ResumenDiario(
        Map<String, Long> afluenciaPorEstacion,
        Map<String, Long> usuariosUnicosPorEstacion,
        Map<Integer, Long> flujoPorHora,
        List<Conteo<Integer>> horasPico,
        List<Conteo<String>> rutasMasUtilizadas,
        Map<String, List<String>> patronesDeViaje,
        List<TiempoTramo> tiemposPorTramo,
        double minutosPromedioGeneral,
        List<EstadoRuta> estadoDeRutas) {
}
```

src/tecnomovil/analisis/AnalisisTransporte.java

```
package tecnomovil.analisis;

import java.time.Duration;
import java.util.Collection;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.LongSummaryStatistics;
import java.util.Map;
import java.util.TreeMap;
import java.util.function.DoubleFunction;
import java.util.function.Function;
import java.util.function.Predicate;
import java.util.function.ToIntFunction;
import java.util.stream.Collectors;
import java.util.stream.IntStream;
import java.util.stream.Stream;
import tecnomovil.modelo.Conteo;
import tecnomovil.modelo.EstadoRuta;
import tecnomovil.modelo.NivelOcupacion;
import tecnomovil.modelo.RegistroTransporte;
import tecnomovil.modelo.ResumenDiario;
import tecnomovil.modelo.TiempoTramo;

/**
 * Módulo funcional de procesamiento y agregación de datos diarios de TecnoMovil Data.
 *
 * <p>Cada método público es una función pura: su resultado depende solo de sus parámetros, no
 * modifica la lista recibida, no toca estado compartido y devuelve colecciones inmutables.
 * El mismo código funciona en secuencial o en paralelo porque ninguna lambda comparte estado.
 */
public record AnalisisTransporte(boolean paralelo) {

    private static final Predicate<RegistroTransporte> TODOS = registro -> true;
    private static final Predicate<RegistroTransporte> ES_ENTRADA = RegistroTransporte::esEntrada;

    /** Desplazamiento de un usuario entre dos estaciones, medido en segundos. */
    private record Desplazamiento(Tramo tramo, long segundos) {
    }

    /** Par origen-destino que identifica un tramo. */
    private record Tramo(String origen, String destino) {
    }

    public static AnalisisTransporte secuencial() {
        return new AnalisisTransporte(false);
    }

    public static AnalisisTransporte enParalelo() {
        return new AnalisisTransporte(true);
    }

    private <T> Stream<T> flujo(Collection<T> datos) {
        return paralelo ? datos.parallelStream() : datos.stream();
    }

    // ------------------------------------------------------------------ a) Afluencia por estación

    /** Número de ingresos registrados en cada estación durante el día. */
    public Map<String, Long> afluenciaPorEstacion(List<RegistroTransporte> registros) {
        return contarPor(registros, ES_ENTRADA, RegistroTransporte::estacion);
    }

    /** Número de usuarios distintos que ingresaron a cada estación (un usuario cuenta una vez). */
    public Map<String, Long> usuariosUnicosPorEstacion(List<RegistroTransporte> registros) {
        return flujo(registros)
                .filter(ES_ENTRADA)
```

```
            .collect(Collectors.collectingAndThen(
                    Collectors.groupingBy(
                            RegistroTransporte::estacion,
                            TreeMap::new,
                            Collectors.collectingAndThen(
                                    Collectors.mapping(RegistroTransporte::idUsuario, Collectors.toSet()),
                                    usuarios -> (long) usuarios.size())),
                    Collections::unmodifiableMap));
}

// ------------------------------------------------------------- b) Horas pico

/** Número de registros (entradas y salidas) por cada hora del día. */
public Map<Integer, Long> flujoPorHora(List<RegistroTransporte> registros) {
    return contarPor(registros, TODOS, RegistroTransporte::hora);
}

/** Las {@code cantidad} horas de mayor flujo, de mayor a menor. */
public List<Conteo<Integer>> horasPico(List<RegistroTransporte> registros, int cantidad) {
    return rankear(flujoPorHora(registros), cantidad);
}

// ------------------------------------------------------------- c) Rutas más utilizadas

/** Rutas ordenadas de mayor a menor volumen de uso. Cada ingreso cuenta como un uso de la ruta. */
public List<Conteo<String>> rutasMasUtilizadas(List<RegistroTransporte> registros) {
    return rankear(contarPor(registros, ES_ENTRADA, RegistroTransporte::ruta), Integer.MAX_VALUE);
}

// ------------------------------------------------------------- d) Patrones de viaje

/** Para cada usuario, las estaciones visitadas en el orden en que aparecen en los registros. */
public Map<String, List<String>> patronesDeViaje(List<RegistroTransporte> registros) {
    return flujo(registros)
            .collect(Collectors.collectingAndThen(
                    Collectors.groupingBy(
                            RegistroTransporte::idUsuario,
                            TreeMap::new,
                            Collectors.mapping(RegistroTransporte::estacion,
                                    Collectors.toUnmodifiableList())),
                    Collections::unmodifiableMap));
}

// ------------------------------------------------------------- e) Tiempo entre estaciones

/** Tiempo promedio de cada tramo origen-destino, del más demorado al más rápido. */
public List<TiempoTramo> tiemposPorTramo(List<RegistroTransporte> registros) {
    Map<Tramo, LongSummaryStatistics> estadisticas = desplazamientos(registros)
            .collect(Collectors.groupingBy(
                    Desplazamiento::tramo,
                    Collectors.summarizingLong(Desplazamiento::segundos)));

    return estadisticas.entrySet().stream()
            .map(tramo -> new TiempoTramo(
                    tramo.getKey().origen(),
                    tramo.getKey().destino(),
                    tramo.getValue().getCount(),
                    tramo.getValue().getAverage() / 60.0))
            .sorted(Comparator.comparingDouble(TiempoTramo::minutosPromedio).reversed()
                    .thenComparing(TiempoTramo::origen)
                    .thenComparing(TiempoTramo::destino))
            .toList();
}

/** Tiempo promedio, en minutos, de todos los desplazamientos del día. */
public double minutosPromedioGeneral(List<RegistroTransporte> registros) {
    return desplazamientos(registros)
            .mapToLong(Desplazamiento::segundos)
            .average()
            .orElse(0.0) / 60.0;
```

```
}

private Stream<Desplazamiento> desplazamientos(List<RegistroTransporte> registros) {
    Map<String, List<RegistroTransporte>> porUsuario =
            flujo(registros).collect(Collectors.groupingBy(RegistroTransporte::idUsuario));
    return flujo(porUsuario.values()).flatMap(AnalisisTransporte::desplazamientosDeUsuario);
}

/**
 * Recorre los registros consecutivos de un usuario. Solo una entrada seguida de una salida en otra
 * estación es un desplazamiento: una salida seguida de una entrada es tiempo fuera del sistema
 * (por ejemplo, la jornada de trabajo) y falsearía el promedio.
 */
private static Stream<Desplazamiento> desplazamientosDeUsuario(List<RegistroTransporte> registrosUsuario)
        {
    List<RegistroTransporte> ordenados = registrosUsuario.stream()
            .sorted(Comparator.comparing(RegistroTransporte::timestamp))
            .toList();

    return IntStream.range(0, ordenados.size() - 1)
            .filter(i -> esDesplazamiento(ordenados.get(i), ordenados.get(i + 1)))
            .mapToObj(i -> crearDesplazamiento(ordenados.get(i), ordenados.get(i + 1)));
}

private static boolean esDesplazamiento(RegistroTransporte actual, RegistroTransporte siguiente) {
    return actual.esEntrada()
            && !siguiente.esEntrada()
            && !actual.estacion().equals(siguiente.estacion());
}

private static Desplazamiento crearDesplazamiento(RegistroTransporte origen, RegistroTransporte destino) {
    return new Desplazamiento(
            new Tramo(origen.estacion(), destino.estacion()),
            Duration.between(origen.timestamp(), destino.timestamp()).toSeconds());
}

// ------------------------------------------------------------------ f) Sobrecarga en rutas

/**
 * Evalúa cada ruta en su hora de mayor demanda y la marca como crítica si supera el umbral.
 *
 * @param capacidadDe función que entrega la capacidad horaria simulada de una ruta
 * @param umbral      ocupación a partir de la cual la ruta es crítica (0.90 equivale a 90 %)
 */
public List<EstadoRuta> detectarSobrecarga(
        List<RegistroTransporte> registros,
        ToIntFunction<String> capacidadDe,
        double umbral) {

    Map<String, Map<Integer, Long>> ingresosPorRutaYHora = flujo(registros)
            .filter(ES_ENTRADA)
            .collect(Collectors.groupingBy(
                    RegistroTransporte::ruta,
                    Collectors.groupingBy(RegistroTransporte::hora, Collectors.counting())));

    DoubleFunction<NivelOcupacion> clasificar = clasificador(umbral);

    return ingresosPorRutaYHora.entrySet().stream()
            .map(ruta -> evaluarRuta(
                    ruta.getKey(),
                    ruta.getValue(),
                    capacidadDe.applyAsInt(ruta.getKey()),
                    clasificar))
            .sorted(Comparator.comparingDouble(EstadoRuta::ocupacion).reversed()
                    .thenComparing(EstadoRuta::ruta))
            .toList();
}

/** Función de orden superior: recibe un umbral y devuelve la función que clasifica una ocupación. */
static DoubleFunction<NivelOcupacion> clasificador(double umbral) {
```

```
    return ocupacion -> ocupacion > umbral ? NivelOcupacion.CRITICA : NivelOcupacion.NORMAL;
}

private static EstadoRuta evaluarRuta(
        String ruta,
        Map<Integer, Long> ingresosPorHora,
        int capacidadHoraria,
        DoubleFunction<NivelOcupacion> clasificar) {

    Map.Entry<Integer, Long> horaMaxima = ingresosPorHora.entrySet().stream()
            .max(Map.Entry.<Integer, Long>comparingByValue()
                    .thenComparing(Map.Entry.comparingByKey(Comparator.reverseOrder())))
            .orElseThrow();

    double ocupacion = (double) horaMaxima.getValue() / capacidadHoraria;

    return new EstadoRuta(
            ruta,
            horaMaxima.getKey(),
            horaMaxima.getValue(),
            capacidadHoraria,
            ocupacion,
            clasificar.apply(ocupacion));
}

// ---------------------------------------------------------------- Resumen del día

/** Ejecuta las seis tareas y devuelve sus respuestas en un solo valor inmutable. */
public ResumenDiario resumir(
        List<RegistroTransporte> registros,
        int cantidadHorasPico,
        ToIntFunction<String> capacidadDe,
        double umbral) {

    return new ResumenDiario(
            afluenciaPorEstacion(registros),
            usuariosUnicosPorEstacion(registros),
            flujoPorHora(registros),
            horasPico(registros, cantidadHorasPico),
            rutasMasUtilizadas(registros),
            patronesDeViaje(registros),
            tiemposPorTramo(registros),
            minutosPromedioGeneral(registros),
            detectarSobrecarga(registros, capacidadDe, umbral));
}

// ---------------------------------------------------------------- Funciones reutilizables

/** Función de orden superior: el criterio de filtrado y la clave de agrupación llegan como parámetros. */
private <K extends Comparable<K>> Map<K, Long> contarPor(
        List<RegistroTransporte> registros,
        Predicate<RegistroTransporte> filtro,
        Function<RegistroTransporte, K> clave) {

    return flujo(registros)
            .filter(filtro)
            .collect(Collectors.collectingAndThen(
                    Collectors.groupingBy(clave, TreeMap::new, Collectors.counting()),
                    Collections::unmodifiableMap));
}

/** Convierte un mapa de totales en una lista ordenada de mayor a menor (desempate por clave). */
private static <K extends Comparable<K>> List<Conteo<K>> rankear(Map<K, Long> totales, int cantidad) {
    Comparator<Conteo<K>> mayorTotalPrimero =
            Comparator.comparingLong((Conteo<K> conteo) -> conteo.total()).reversed()
                    .thenComparing(Conteo::clave);

    return totales.entrySet().stream()
            .map(total -> new Conteo<>(total.getKey(), total.getValue()))
            .sorted(mayorTotalPrimero)
```

```
                .limit(cantidad)
                .toList();
    }
}
```

src/tecnomovil/datos/CatalogoRed.java

```
package tecnomovil.datos;

import java.util.List;
import java.util.Map;
import java.util.function.IntUnaryOperator;
import java.util.function.ToIntFunction;
import java.util.stream.Collectors;
import java.util.stream.IntStream;

/** Red simulada de TecnoValle: rutas, estaciones, popularidad y capacidad horaria. */
public final class CatalogoRed {

    /** Pasajeros diarios que menciona el caso de estudio; la capacidad base está dimensionada para ellos. */
    public static final int USUARIOS_DE_REFERENCIA = 450_000;

    private static final int CAPACIDAD_RUTA_DESCONOCIDA = 10_000;

    /** Ruta de la red. {@code peso} indica qué tan popular es frente a las demás. */
    public record Ruta(String codigo, List<String> estaciones, int peso, int capacidadHoraria, int
            minutosPorParada) {
    }

    public static final List<Ruta> RUTAS = List.of(
            new Ruta("T1 Norte-Sur",
                    List.of("Portal Norte", "Alameda", "Universidad", "Centro", "Hospital", "Parque Sur",
                            "Portal Sur"),
                    34, 50_000, 5),
            new Ruta("T2 Oriente-Occidente",
                    List.of("Portal Oriente", "La Ceiba", "Centro", "Mercado", "Estadio", "Portal Occidente"),
                    26, 44_000, 6),
            new Ruta("C1 Circular Centro",
                    List.of("Centro", "Alcaldía", "Terminal", "Universidad"),
                    18, 36_000, 4),
            new Ruta("A1 Alimentadora Norte",
                    List.of("Portal Norte", "Los Pinos", "Villa Nueva", "El Mirador"),
                    12, 26_000, 7),
            new Ruta("A2 Alimentadora Sur",
                    List.of("Portal Sur", "El Prado", "San José", "La Esperanza"),
                    10, 24_000, 8));

    /** Rutas repetidas según su peso: elegir una posición al azar respeta la popularidad. */
    static final List<Ruta> RUTAS_PONDERADAS = RUTAS.stream()
            .flatMap(ruta -> IntStream.range(0, ruta.peso()).mapToObj(i -> ruta))
            .toList();

    private static final Map<String, Integer> CAPACIDAD_BASE = RUTAS.stream()
            .collect(Collectors.toUnmodifiableMap(Ruta::codigo, Ruta::capacidadHoraria));

    private CatalogoRed() {
    }

    /**
     * Devuelve la función de capacidad para un volumen de usuarios, de modo que la ocupación
     * simulada no cambie al procesar más o menos datos que los del día de referencia.
     */
    public static ToIntFunction<String> capacidadPara(int usuarios) {
        IntUnaryOperator escalar = capacidad ->
                Math.max(1, (int) Math.round((double) capacidad * usuarios / USUARIOS_DE_REFERENCIA));
        return ruta -> escalar.applyAsInt(CAPACIDAD_BASE.getOrDefault(ruta, CAPACIDAD_RUTA_DESCONOCIDA));
    }
}
```

src/tecnomovil/datos/GeneradorDatos.java

```
package tecnomovil.datos;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.Comparator;
import java.util.List;
import java.util.SplittableRandom;
import java.util.stream.IntStream;
import java.util.stream.Stream;
import tecnomovil.datos.CatalogoRed.Ruta;
import tecnomovil.modelo.Accion;
import tecnomovil.modelo.RegistroTransporte;

/**
 * Genera un día simulado de registros.
 *
 * <p>Es una función pura: con los mismos parámetros siempre produce los mismos datos. Cada viaje
 * crea su propio generador pseudoaleatorio a partir de una semilla derivada, así ningún hilo
 * comparte estado y la generación se puede paralelizar sin alterar el resultado.
 */
public final class GeneradorDatos {

    private static final List<Integer> HORAS_MANANA = repetirHoras(5, 2, 6, 6, 7, 9, 8, 6, 9, 2);
    private static final List<Integer> HORAS_MEDIODIA = repetirHoras(11, 3, 12, 5, 13, 4, 14, 2);
    private static final List<Integer> HORAS_TARDE = repetirHoras(16, 3, 17, 8, 18, 9, 19, 4, 20, 1);

    /** Jornadas de cada viaje según cuántos viajes hace el usuario en el día. */
    private static final List<List<List<Integer>>> JORNADAS = List.of(
            List.of(HORAS_MANANA),
            List.of(HORAS_MANANA, HORAS_TARDE),
            List.of(HORAS_MANANA, HORAS_MEDIODIA, HORAS_TARDE));

    /** Viajes diarios posibles: la mayoría de usuarios hace ida y regreso. */
    private static final List<Integer> VIAJES_POR_USUARIO = List.of(1, 2, 2, 2, 2, 3);

    private GeneradorDatos() {
    }

    public static List<RegistroTransporte> generar(int usuarios, long semilla, LocalDate fecha) {
        return IntStream.range(0, usuarios)
                .parallel()
                .boxed()
                .flatMap(usuario -> registrosDeUsuario(usuario, semilla, fecha))
                .sorted(Comparator.comparing(RegistroTransporte::timestamp))
                .toList();
    }

    private static Stream<RegistroTransporte> registrosDeUsuario(int usuario, long semilla, LocalDate fecha) {
        SplittableRandom azar = new SplittableRandom(semillaDerivada(semilla, usuario, 0));
        Ruta ruta = elegir(CatalogoRed.RUTAS_PONDERADAS, azar);
        int viajes = elegir(VIAJES_POR_USUARIO, azar);
        int casa = azar.nextInt(ruta.estaciones().size());
        int trabajo = paradaDistinta(casa, ruta, azar);
        int diligencia = paradaDistinta(trabajo, ruta, azar);
        List<Integer> paradas = recorrido(viajes, casa, trabajo, diligencia);
        List<List<Integer>> jornadas = JORNADAS.get(viajes - 1);

        return IntStream.range(0, viajes)
                .boxed()
                .flatMap(viaje -> registrosDeViaje(
                        "U" + String.format("%07d", usuario),
                        ruta,
                        paradas.get(viaje),
                        paradas.get(viaje + 1),
                        jornadas.get(viaje),
                        fecha,
                        new SplittableRandom(semillaDerivada(semilla, usuario, viaje + 1))));
```

```
    }

    private static List<Integer> recorrido(int viajes, int casa, int trabajo, int diligencia) {
        return viajes == 1 ? List.of(casa, trabajo)
                : viajes == 2 ? List.of(casa, trabajo, casa)
                : diligencia == casa ? List.of(casa, trabajo, casa, trabajo)
                : List.of(casa, trabajo, diligencia, casa);
    }

    private static Stream<RegistroTransporte> registrosDeViaje(
            String idUsuario,
            Ruta ruta,
            int origen,
            int destino,
            List<Integer> horas,
            LocalDate fecha,
            SplittableRandom azar) {

        LocalDateTime entrada = fecha.atTime(elegir(horas, azar), azar.nextInt(60), azar.nextInt(60));
        int paradasRecorridas = Math.abs(destino - origen);
        long segundos = paradasRecorridas * (ruta.minutosPorParada() * 60L + azar.nextInt(-45, 91));

        return Stream.of(
                new RegistroTransporte(idUsuario, ruta.codigo(), ruta.estaciones().get(origen),
                        Accion.ENTRADA, entrada),
                new RegistroTransporte(idUsuario, ruta.codigo(), ruta.estaciones().get(destino),
                        Accion.SALIDA,
                        entrada.plusSeconds(segundos)));
    }

    private static int paradaDistinta(int parada, Ruta ruta, SplittableRandom azar) {
        int total = ruta.estaciones().size();
        return (parada + 1 + azar.nextInt(total - 1)) % total;
    }

    private static <T> T elegir(List<T> opciones, SplittableRandom azar) {
        return opciones.get(azar.nextInt(opciones.size()));
    }

    private static long semillaDerivada(long semilla, int usuario, int viaje) {
        return semilla + usuario * 31L + (long) viaje * 1_000_003L * (usuario + 1L);
    }

    /** Recibe pares hora-peso y devuelve cada hora repetida tantas veces como indique su peso. */
    private static List<Integer> repetirHoras(int... horasYPesos) {
        return IntStream.iterate(0, i -> i < horasYPesos.length, i -> i + 2)
                .boxed()
                .flatMap(i -> IntStream.range(0, horasYPesos[i + 1]).mapToObj(n -> horasYPesos[i]))
                .toList();
    }
}
```

src/tecnomovil/reporte/Reporte.java

```
package tecnomovil.reporte;

import java.util.Collections;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.stream.Collectors;
import java.util.stream.Stream;
import tecnomovil.modelo.EstadoRuta;
import tecnomovil.modelo.NivelOcupacion;
import tecnomovil.modelo.ResumenDiario;
import tecnomovil.modelo.TiempoTramo;

/** Convierte resultados en texto. Son funciones puras: devuelven cadenas y no imprimen nada. */
public final class Reporte {

    private static final Locale COLOMBIA = Locale.forLanguageTag("es-CO");
    private static final String LINEA = "=".repeat(86);
    private static final int ANCHO_BARRA = 40;
    private static final int TRAMOS_A_MOSTRAR = 5;
    private static final int USUARIOS_DE_EJEMPLO = 5;

    private Reporte() {
    }

    public static String titulo(String texto) {
        return String.join("\n", LINEA, "  " + texto, LINEA);
    }

    public static String encabezado(long registros, long usuarios) {
        return titulo("TECNOMOVIL DATA - Procesamiento funcional de un día de transporte")
                + "\n" + String.format(COLOMBIA, "Registros procesados: %,d | Usuarios simulados: %,d",
                        registros, usuarios);
    }

    public static String afluencia(ResumenDiario resumen) {
        String filas = resumen.afluenciaPorEstacion().entrySet().stream()
                .sorted(Map.Entry.<String, Long>comparingByValue(Collections.reverseOrder())
                        .thenComparing(Map.Entry.comparingByKey()))
                .map(estacion -> String.format(COLOMBIA, "  %-20s %,12d %,16d",
                        estacion.getKey(),
                        estacion.getValue(),
                        resumen.usuariosUnicosPorEstacion().get(estacion.getKey())))
                .collect(Collectors.joining("\n"));

        return seccion("a) Afluencia por estación",
                String.format("  %-20s %12s %16s", "Estación", "Ingresos", "Usuarios únicos"), filas);
    }

    public static String horasPico(ResumenDiario resumen) {
        long maximo = resumen.flujoPorHora().values().stream().mapToLong(Long::longValue).max().orElse(1);

        String histograma = resumen.flujoPorHora().entrySet().stream()
                .map(hora -> String.format(COLOMBIA, "  %02d:00 %,10d %s",
                        hora.getKey(),
                        hora.getValue(),
                        "".repeat((int) (hora.getValue() * ANCHO_BARRA / maximo))))
                .collect(Collectors.joining("\n"));

        String pico = resumen.horasPico().stream()
                .map(hora -> String.format(COLOMBIA, "%02d:00 (%,d)", hora.clave(), hora.total()))
                .collect(Collectors.joining(", "));

        return seccion("b) Horas pico (registros por hora)", histograma, "  Horas pico: " + pico);
    }

    public static String rutas(ResumenDiario resumen) {
```

```
    long total = resumen.rutasMasUtilizadas().stream().mapToLong(ruta -> ruta.total()).sum();

    String filas = resumen.rutasMasUtilizadas().stream()
            .map(ruta -> String.format(COLOMBIA, "  %-24s %,12d %8.1f %%",
                    ruta.clave(), ruta.total(), 100.0 * ruta.total() / Math.max(total, 1)))
            .collect(Collectors.joining("\n"));

    return seccion("c) Rutas más utilizadas",
            String.format("  %-24s %12s %10s", "Ruta", "Ingresos", "Part."), filas);
}

public static String patrones(ResumenDiario resumen) {
    String ejemplos = resumen.patronesDeViaje().entrySet().stream()
            .limit(USUARIOS_DE_EJEMPLO)
            .map(usuario -> "  " + usuario.getKey() + ": " + String.join(" > ", usuario.getValue()))
            .collect(Collectors.joining("\n"));

    return seccion("d) Patrones de viaje por usuario",
            String.format(COLOMBIA, "  Usuarios con patrón calculado: %,d. Primeros %d:",
                    resumen.patronesDeViaje().size(), USUARIOS_DE_EJEMPLO),
            ejemplos);
}

public static String tiempos(ResumenDiario resumen) {
    List<TiempoTramo> tramos = resumen.tiemposPorTramo();

    String filas = tramos.stream()
            .limit(TRAMOS_A_MOSTRAR)
            .map(tramo -> String.format(COLOMBIA, "  %-40s %,10d %9.1f min",
                    tramo.origen() + " > " + tramo.destino(), tramo.viajes(), tramo.minutosPromedio()))
            .collect(Collectors.joining("\n"));

    return seccion("e) Tiempo promedio entre estaciones",
            String.format(COLOMBIA, "  Promedio general: %.1f min | Tramos distintos: %,d. Los %d más
                    demorados:",
                    resumen.minutosPromedioGeneral(), tramos.size(), TRAMOS_A_MOSTRAR),
            String.format("  %-40s %10s %13s", "Tramo", "Viajes", "Promedio"),
            filas);
}

public static String sobrecarga(ResumenDiario resumen, double umbral) {
    String filas = resumen.estadoDeRutas().stream()
            .map(Reporte::filaEstado)
            .collect(Collectors.joining("\n"));

    return seccion(String.format(COLOMBIA, "f) Sobrecarga en rutas (crítica si supera %.0f %% de
            ocupación)", umbral * 100),
            String.format("  %-24s %6s %10s %10s %7s  %s", "Ruta", "Hora", "Pasajeros", "Capacidad",
                    "Ocup.", "Estado"),
            filas);
}

private static String filaEstado(EstadoRuta ruta) {
    return String.format(COLOMBIA, "  %-24s %02d:00 %,10d %,10d %6.1f%%  %s",
            ruta.ruta(), ruta.horaMaxima(), ruta.pasajerosHoraMaxima(), ruta.capacidadHoraria(),
            ruta.ocupacion() * 100,
            ruta.nivel() == NivelOcupacion.CRITICA ? "CRÍTICA" : "Normal");
}

public static String rendimiento(
        long registros, long milisSecuencial, long milisParalelo, int nucleos, boolean mismosResultados) {
    return seccion("Secuencial frente a parallelStream() (mejor de 3 corridas)",
            String.format(COLOMBIA, "  Registros procesados: %,d", registros),
            String.format(COLOMBIA, "  Secuencial: %,d ms | Paralelo: %,d ms | Núcleos: %d | Aceleración:
                    %.2fx",
                    milisSecuencial, milisParalelo, nucleos, (double) milisSecuencial /
                            Math.max(milisParalelo, 1)),
            "  ¿Ambos modos dieron exactamente el mismo resultado? " + (mismosResultados ? "SÍ" : "NO"));
}
```

```
    public static String partes(String... textos) {
        return Stream.of(textos).collect(Collectors.joining("\n\n"));
    }

    private static String seccion(String titulo, String... lineas) {
        return Stream.concat(Stream.of(titulo, "-".repeat(86)), Stream.of(lineas))
                .collect(Collectors.joining("\n"));
    }
}
```

src/tecnomovil/Main.java

```
package tecnomovil;

import java.time.LocalDate;
import java.util.Comparator;
import java.util.List;
import java.util.function.Supplier;
import java.util.function.ToIntFunction;
import java.util.stream.IntStream;
import tecnomovil.analisis.AnalisisTransporte;
import tecnomovil.datos.CatalogoRed;
import tecnomovil.datos.GeneradorDatos;
import tecnomovil.modelo.RegistroTransporte;
import tecnomovil.modelo.ResumenDiario;
import tecnomovil.reporte.Reporte;

/**
 * Punto de entrada. Toda la lógica está en funciones puras; esta clase es el único lugar con
 * efectos secundarios: lee los argumentos, mide el reloj e imprime en consola.
 *
 * <p>Uso: {@code java tecnomovil.Main [usuarios] [parte]}, donde parte es 1, 2 o todo.
 */
public final class Main {

    private static final int USUARIOS_POR_DEFECTO = CatalogoRed.USUARIOS_DE_REFERENCIA;
    private static final long SEMILLA = 2026L;
    private static final LocalDate FECHA = LocalDate.of(2026, 10, 30);
    private static final int HORAS_PICO = 3;
    private static final double UMBRAL_CRITICO = 0.90;
    private static final int CORRIDAS = 3;

    private record Medicion<T>(T resultado, long milisegundos) {
    }

    private Main() {
    }

    public static void main(String[] args) {
        int usuarios = args.length > 0 ? Integer.parseInt(args[0]) : USUARIOS_POR_DEFECTO;
        String parte = args.length > 1 ? args[1] : "todo";

        List<RegistroTransporte> registros = GeneradorDatos.generar(usuarios, SEMILLA, FECHA);
        ToIntFunction<String> capacidad = CatalogoRed.capacidadPara(usuarios);

        Medicion<ResumenDiario> secuencial = mejorCorrida(
                () -> AnalisisTransporte.secuencial().resumir(registros, HORAS_PICO, capacidad,
                        UMBRAL_CRITICO));
        Medicion<ResumenDiario> paralelo = mejorCorrida(
                () -> AnalisisTransporte.enParalelo().resumir(registros, HORAS_PICO, capacidad,
                        UMBRAL_CRITICO));

        ResumenDiario resumen = secuencial.resultado();
        String primeraParte = Reporte.partes(
                Reporte.encabezado(registros.size(), usuarios),
                Reporte.afluencia(resumen),
                Reporte.horasPico(resumen));
        String segundaParte = Reporte.partes(
                Reporte.rutas(resumen),
                Reporte.patrones(resumen),
                Reporte.tiempos(resumen),
                Reporte.sobrecarga(resumen, UMBRAL_CRITICO),
                Reporte.rendimiento(
                        registros.size(),
                        secuencial.milisegundos(),
                        paralelo.milisegundos(),
                        Runtime.getRuntime().availableProcessors(),
                        resumen.equals(paralelo.resultado())));
```

```
        System.out.println(switch (parte) {
            case "1" -> primeraParte;
            case "2" -> segundaParte;
            default -> Reporte.partes(primeraParte, segundaParte);
        });
    }

    private static <T> Medicion<T> mejorCorrida(Supplier<T> tarea) {
        return IntStream.range(0, CORRIDAS)
                .mapToObj(corrida -> medir(tarea))
                .min(Comparator.comparingLong(Medicion::milisegundos))
                .orElseThrow();
    }

    private static <T> Medicion<T> medir(Supplier<T> tarea) {
        long inicio = System.nanoTime();
        T resultado = tarea.get();
        return new Medicion<>(resultado, (System.nanoTime() - inicio) / 1_000_000);
    }
}
```

src/tecnomovil/verificacion/VerificacionFunciones.java

```
package tecnomovil.verificacion;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.function.BooleanSupplier;
import java.util.function.ToIntFunction;
import java.util.stream.Collectors;
import java.util.stream.Stream;
import tecnomovil.analisis.AnalisisTransporte;
import tecnomovil.datos.CatalogoRed;
import tecnomovil.datos.GeneradorDatos;
import tecnomovil.modelo.Accion;
import tecnomovil.modelo.Conteo;
import tecnomovil.modelo.NivelOcupacion;
import tecnomovil.modelo.RegistroTransporte;

/**
 * Comprueba los resultados contra un conjunto pequeño calculado a mano, además de la pureza,
 * la inmutabilidad y la equivalencia entre el modo secuencial y el paralelo. No usa frameworks.
 */
public final class VerificacionFunciones {

    private static final String T1 = "T1";
    private static final String T2 = "T2";
    private static final double TOLERANCIA = 1e-9;

    private record Comprobacion(String nombre, boolean aprobada) {
    }

    private static final List<RegistroTransporte> DATOS = List.of(
            registro("U1", T1, "Portal Norte", Accion.ENTRADA, 7, 0),
            registro("U2", T1, "Portal Norte", Accion.ENTRADA, 7, 10),
            registro("U1", T1, "Centro", Accion.SALIDA, 7, 20),
            registro("U2", T1, "Universidad", Accion.SALIDA, 7, 25),
            registro("U3", T2, "Mercado", Accion.ENTRADA, 7, 40),
            registro("U3", T2, "Centro", Accion.SALIDA, 7, 50),
            registro("U1", T1, "Centro", Accion.ENTRADA, 17, 0),
            registro("U1", T1, "Portal Norte", Accion.SALIDA, 17, 30));

    private static final ToIntFunction<String> CAPACIDAD = ruta -> ruta.equals(T1) ? 2 : 10;

    private VerificacionFunciones() {
    }

    public static void main(String[] args) {
        List<Comprobacion> comprobaciones = comprobaciones();

        System.out.println(comprobaciones.stream()
                .map(c -> (c.aprobada() ? "[ OK ] " : "[FALLA] ") + c.nombre())
                .collect(Collectors.joining("\n")));

        long aprobadas = comprobaciones.stream().filter(Comprobacion::aprobada).count();
        System.out.printf("%nComprobaciones aprobadas: %d de %d%n", aprobadas, comprobaciones.size());

        if (aprobadas != comprobaciones.size()) {
            System.exit(1);
        }
    }

    private static List<Comprobacion> comprobaciones() {
        AnalisisTransporte analisis = AnalisisTransporte.secuencial();
        List<RegistroTransporte> grande = GeneradorDatos.generar(20_000, 7L, LocalDate.of(2026, 10, 30));
        ToIntFunction<String> capacidadGrande = CatalogoRed.capacidadPara(20_000);
```

```
    return Stream.of(
            comprobar("a) Afluencia: Portal Norte 2, Centro 1, Mercado 1", () ->
                    analisis.afluenciaPorEstacion(DATOS)
                            .equals(Map.of("Portal Norte", 2L, "Centro", 1L, "Mercado", 1L))),
            comprobar("a) Un usuario que entra dos veces cuenta una sola vez como único", () ->
                    analisis.usuariosUnicosPorEstacion(duplicarDatos())
                            .equals(Map.of("Portal Norte", 2L, "Centro", 1L, "Mercado", 1L))),
            comprobar("b) Flujo: 6 registros a las 7:00 y 2 a las 17:00", () ->
                    analisis.flujoPorHora(DATOS).equals(Map.of(7, 6L, 17, 2L))),
            comprobar("b) La hora pico es las 7:00", () ->
                    analisis.horasPico(DATOS, 1).equals(List.of(new Conteo<>(7, 6L)))),
            comprobar("c) Rutas: T1 con 3 ingresos antes que T2 con 1", () ->
                    analisis.rutasMasUtilizadas(DATOS)
                            .equals(List.of(new Conteo<>(T1, 3L), new Conteo<>(T2, 1L)))),
            comprobar("d) Patrón de U1 en el orden de los registros", () ->
                    analisis.patronesDeViaje(DATOS).get("U1")
                            .equals(List.of("Portal Norte", "Centro", "Centro", "Portal Norte"))),
            comprobar("e) Promedio general: (20 + 15 + 10 + 30) / 4 = 18,75 min", () ->
                    Math.abs(analisis.minutosPromedioGeneral(DATOS) - 18.75) < TOLERANCIA),
            comprobar("e) Se excluye la estadía de U1 entre las 7:20 y las 17:00", () ->
                    analisis.tiemposPorTramo(DATOS).size() == 4
                            && analisis.tiemposPorTramo(DATOS).stream().allMatch(t -> t.minutosPromedio()
                                    <= 30)),
            comprobar("f) T1 (2 de 2 = 100 %) es crítica", () ->
                    analisis.detectarSobrecarga(DATOS, CAPACIDAD, 0.90).get(0).nivel() ==
                            NivelOcupacion.CRITICA),
            comprobar("f) T2 (1 de 10 = 10 %) es normal", () ->
                    analisis.detectarSobrecarga(DATOS, CAPACIDAD, 0.90).get(1).nivel() ==
                            NivelOcupacion.NORMAL),
            comprobar("Pureza: dos llamadas con la misma entrada dan el mismo resultado", () ->
                    analisis.resumir(DATOS, 3, CAPACIDAD, 0.90).equals(analisis.resumir(DATOS, 3,
                            CAPACIDAD, 0.90))),
            comprobar("Inmutabilidad: la lista original queda igual tras procesarla", () -> {
                List<RegistroTransporte> copia = List.copyOf(DATOS);
                analisis.resumir(DATOS, 3, CAPACIDAD, 0.90);
                return DATOS.equals(copia);
            }),
            comprobar("Inmutabilidad: una lista mutable de entrada tampoco se altera", () -> {
                List<RegistroTransporte> mutable = new ArrayList<>(DATOS);
                analisis.resumir(mutable, 3, CAPACIDAD, 0.90);
                return mutable.equals(DATOS);
            }),
            comprobar("Inmutabilidad: el mapa de afluencia rechaza modificaciones", () ->
                    rechazaCambios(() -> analisis.afluenciaPorEstacion(DATOS).put("X", 1L))),
            comprobar("Inmutabilidad: el mapa de patrones y sus listas rechazan modificaciones", () ->
                    rechazaCambios(() -> analisis.patronesDeViaje(DATOS).clear())
                            && rechazaCambios(() -> analisis.patronesDeViaje(DATOS).get("U1").add("X"))),
            comprobar("Inmutabilidad: el ranking de rutas rechaza modificaciones", () ->
                    rechazaCambios(() -> analisis.rutasMasUtilizadas(DATOS).clear())),
            comprobar("Lista vacía: no falla y devuelve resultados vacíos", () ->
                    analisis.resumir(List.of(), 3, CAPACIDAD, 0.90).afluenciaPorEstacion().isEmpty()
                            && analisis.minutosPromedioGeneral(List.of()) == 0.0),
            comprobar("Generador: con la misma semilla produce los mismos datos", () ->
                    grande.equals(GeneradorDatos.generar(20_000, 7L, LocalDate.of(2026, 10, 30)))),
            comprobar("Paralelismo: parallelStream() da el mismo resultado que stream()", () ->
                    AnalisisTransporte.secuencial().resumir(grande, 3, capacidadGrande, 0.90)
                            .equals(AnalisisTransporte.enParalelo().resumir(grande, 3, capacidadGrande,
                                    0.90))))
            .toList();
}

private static List<RegistroTransporte> duplicarDatos() {
    return Stream.concat(DATOS.stream(), DATOS.stream()).toList();
}

private static Comprobacion comprobar(String nombre, BooleanSupplier condicion) {
    return new Comprobacion(nombre, condicion.getAsBoolean());
}

private static boolean rechazaCambios(Runnable modificacion) {
```

```
        try {
            modificacion.run();
            return false;
        } catch (UnsupportedOperationException esperada) {
            return true;
        }
    }

    private static RegistroTransporte registro(
            String usuario, String ruta, String estacion, Accion accion, int hora, int minuto) {
        return new RegistroTransporte(usuario, ruta, estacion, accion, LocalDateTime.of(2026, 10, 30, hora,
                minuto));
    }
}
```